ANN at Scale: Recall vs Latency, Worked Out by Hand

The fundamental dial of vector search — spend more time exploring the index and you find more of the true neighbours — made concrete across HNSW and IVF, with the effort knobs and the diminishing-returns curve they trace.

What this document does. It frames approximate nearest-neighbour search as a single tradeoff between *recall* (did you find the true neighbours, #12) and *latency* (how long it took), then shows how each index family exposes that tradeoff through one knob: `efSearch` for HNSW, `nprobe` for IVF. A worked tradeoff table makes the diminishing-returns shape explicit. Illustrative numbers in Appendix A.

This is an intuition PDF. The exact recall/latency curve is hardware- and data-specific; the *shape* and the knobs are universal.

1 The tradeoff in one sentence

Exact search compares the query to all N vectors — 100% recall, $O(N)$ latency. Approximate search visits only a fraction of them — sub-linear latency, but it can miss true neighbours. Every ANN index is a machine for choosing *how much* of the index to visit, and that choice is a direct exchange:

$$\text{more exploration} \implies \text{higher recall} + \text{higher latency.}$$

There is no setting that gives both perfect recall and minimal latency on a large corpus; you pick a point on the curve.

> **Intuition**
>
> Searching an index is like searching a library for the books closest to a topic. Check every shelf and you are guaranteed to find the best matches, but it takes all day (exact, $O(N)$). Check only the three most promising sections and you are fast but might miss a good book filed oddly (approximate). The knob is "how many sections do I check?" — check more, find more, take longer. Recall and latency are the two ends of that one lever.

2 Where the knob lives in each index

HNSW — `efSearch`. During the greedy graph walk (PDF #9), HNSW keeps a candidate queue of size `efSearch`. Small queue ⇒ it commits early to the first good node it sees and may stop at a local optimum (fast, lower recall). Large queue ⇒ it holds many candidates open and explores more of the graph before settling (slower, higher recall). It is the single runtime recall dial.

IVF — `nprobe`. An inverted-file index clusters the vectors into cells and, at query time, only scans the `nprobe` cells nearest the query. `nprobe`=1 scans one cell (fast, may miss neighbours that fell just over a cell boundary); larger `nprobe` scans more cells (slower, higher recall). Same tradeoff, different mechanism — "how many sections of the library?" literally.

3 The diminishing-returns curve

Raising the effort knob buys recall, but each increment buys *less* than the last — the first few candidates catch the easy neighbours; the stragglers cost ever more exploration. An illustrative HNSW sweep:

`efSearch`	recall@10	latency	Δrecall per 2 × effort
16	0.82	1.0 ms	–
32	0.91	1.7 ms	+0.09
64	0.96	3.0 ms	+0.05
128	0.985	5.6 ms	+0.025
256	0.995	11 ms	+0.010

Doubling effort from 16 → 32 adds 9 points of recall; the same doubling from 128 → 256 adds only 1. Meanwhile latency roughly doubles each step. The knee of the curve — here around `efSearch`= 64–128 — is where most systems sit: nearly all the recall, a fraction of the worst-case latency.

Mini-example · why you stop before 100%

Going from 0.985 to 0.995 recall cost a 2× latency hit (5.6 → 11 ms) for one extra point. Chasing the last 0.5% to reach 1.000 can cost another 5–10×, approaching the $O(N)$ exact cost you were trying to avoid. **The whole value of ANN is conceding a sliver of recall to escape linear scan** — so the right target is "recall high enough that the downstream LLM/re-ranker doesn't notice," not 100%.

4 Tuning it in practice

- **Set a recall target from downstream needs**, not vanity — e.g. "recall@20 ≥ 0.95 so the re-ranker (3.3) has the right candidates." Measure it against exact search on a query sample (#12).
- **Raise the knob** (`efSearch` / `nprobe`) until you hit that target, then stop — everything beyond is latency you are paying for recall nobody uses.
- **Build-time knobs trade too:** larger M / `efConstruction` (HNSW) or more cells (IVF) shift the *whole* curve up, at the cost of memory and index time.
- **Compose with quantization** (#10, #11): PQ shrinks memory so you can hold a bigger/-better graph in RAM — recall, latency, and footprint are a three-way budget.

Equation cheat-sheet

$$\text{exact}: \ \text{recall} = 1, \ O(N); \qquad \text{ANN}: \ \text{recall} \uparrow \ \text{with effort} \uparrow, \ \text{latency} \uparrow$$

$$\text{HNSW knob} = \texttt{efSearch}, \quad \text{IVF knob} = \texttt{nprobe}; \quad \text{tune to a recall target, then stop}$$

Appendix A · Illustrative tradeoff (shape, not a benchmark)

```
# Recall rises and saturates; latency grows ~linearly in effort.
ef      = [16,   32,   64,   128,   256]
recall  = [0.82, 0.91, 0.96, 0.985, 0.995]
latency = [1.0,  1.7,  3.0,  5.6,   11.0]   # ms
# marginal recall per doubling: +0.09, +0.05, +0.025, +0.010  (diminishing)
# pick the knee (~ef 64-128): most of the recall, a fraction of the latency.
```

Internal learning material. Numbers are illustrative of the curve shape; measure your own. v1.0